Core_{sk}

ANONYMOUS AUTHOR(S)

In recent years, significant advances have been made in numerical debugging and static analysis tools. Recent debugging tools significantly reduce overhead using double-double oracles, while recent static analysis tools accurately bound rounding errors using condition numbers. However, these new techniques also have downsides: double-double oracles can miss floating-point errors due to correlated errors, while condition number formalisms cannot cleanly handle over- and underflow. We show that combining both techniques avoids the downsides of each and implement this combination in EasterEgg. EasterEgg uses double-double computations but not as an oracle; instead, it detects erroneous operations using condition numbers, which minimizes overhead while achieving precision and recall similar to more expensive methods. EasterEgg also introduces a separate oracle to detect harmful over- and underflows more accurately; we implement this oracle in a new performant number representation. As a result, EasterEgg achieves a precision of 80.0% and a recall of 96.1% on a collection of 500 difficult numeric benchmarks. An ablation study shows that EasterEgg combines the strengths of both double-double oracles and condition numbers: it is more accurate than double-double oracles yet dramatically faster than methods based on arbitrary-precision condition number computations.

ACM Reference Format:
Anonymous Author(s). 2026. Core_{sk}.1, 1 (*January* 2026), 5 *pages*.

1 Introduction

2 Skia

3 Semantics

We define 2 languages in this section, $\text{Surface}_{\text{Skia}}$ and $\text{Core}_{\text{Skia}}$. $\text{Surface}_{\text{Skia}}$ corresponds to the `SkCanvas` API a user of Skia typically uses to draw pictures.

As this is a description of the Skia API, maybe Draw needs to be specific as in DrawRect etc. and not the generic Geometry Draw

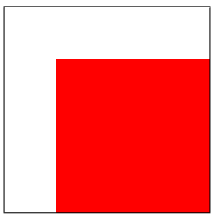
ACM XXXX-XXXX/2026/1-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

$\text{Surface}_{\text{Skia}}$	s	::=	c
Command	c	::=	`Draw(`g`, `p`)`
		|	`Clip(`cl`, Intersect`
		|	`Difference)`
		|	`Save() ; `c` ; Restore()`
		|	`SaveLayer(`p`) ; `c` ; Restore()`
		|	c ; c
DrawGeometry	g	::=	`Rect(...)`
		|	`RRect (...)`
		|	`Path(...)`
		|	...
ClipGeometry	cl	::=	`Rect(...)`
		|	`RRect (...)`
		|	`Path(...)`
Paint	p	::=	`Paint(`f`, `st`, `c_f`, `b`)`
Fill	f	::=	`Color(Pixel)`
		|	`LinearGradient(...)`
		|	...
Style	st	::=	`Solid`
		|	`Stroke`
Filter	c_f	::=	`IdFilter`
		|	`LumaFilter`
		|	...
BlendMode	b	::=	`SrcOver`
		|	`DstIn`
		|	`Src`
		|	...

A $\text{Surface}_{\text{Skia}}$ program maintains 2 types of state, the state of canvas and the clip state. The canvas state describes what is currently drawn on the drawing surface. The clip state constrains the extent of what is drawn on the drawing surface. Consider the program:

```
Draw(Rect(64, 64, 256, 256), RED_PAINT)
```

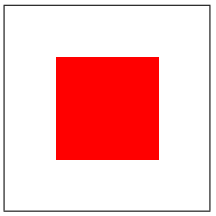
Here we are drawing on a canvas of size 256×256 and `RED_PAINT` being a `Paint` that just fills a geometry with the color Red. This program would render to the following image:



Now consider the program:

```
Clip(Rect(0, 0, 192, 192));
Draw(Rect(64, 64, 256, 256), RED_PAINT)
```

Here we clip to the upper-right corner of the square. The resulting image for rendering this program is:



The contents of `Paint` merit further discussion. `Fills` describe what pixels to fill the geometry with, which could be the same color or a shader describing say a gradient. `Style` describes how the geometry is drawn, whether the extent of the geometry is filled, or only its bounds. `Filters` basically are functions to transform pixels. Finally `BlendModes` describe how the drawn and transformed geometry is *composited* or *blended* onto the canvas. You can consider a blend mode as an operator over pixels.

I ran out of gas here. Probably want pictures for all of this? We cannot assume readers have ever used Skia before, so they do not know what I am on about in the previous paragraph.

It is important to undertsnad that `Clips` are additive to current clip state of the canvas. Just with `Clips` alone you may not reset the clip state. This is taken care of by the `Save(); ...; Restore()` and `SaveLayer(); ...; Restore()` commands. Calling a `Save`/`SaveLayer` is like saving a checkpoint of the current clip state. When a matching `Restore()` is called, all changes made to the clip state are discarded. Internally Skia implements this checkpointing system by managing a stack of clip states, with `Save`/`SaveLayer` pushsing a copy of the current clip state onto the stack, while a `Restore` pops the latest clip state of the stack.

While `Save`/`SaveLayer` both manipulate the clip state of the canvas, `SaveLayer` also allocates a new seperate surface. All commands between a `SaveLayer` and a matching `Restore` affect the new surface. Once `Restore` is called, the new surface is merged onto the old one. How this merge happens is dictated by the `BlendMode` passed to the `Paint` of the `SaveLayer`. This functions just like `Draw` commands. `SaveLayers` can also be used to modify the entire new surface before it is blended back onto the old surface. The `Paint` of a `SaveLayer` can be passed an alpha value, which is multiplied over the entire new surface right before blending. In a similiar vein, a `Filter` inside the `Paint` will be applied over the entire surface before blending. It is important to note that complicated fills like gradients, and fill/stroke styles do not affect how a `SaveLayer` blends its surface onto the previous one.

I dont like old/new/previous here. We always talk about surfaces as bottom and top say for example in the lean formalization. Also probably need to change surfaces to layers here, or layers to surfaces everywhere else.

As our main goal is to try and verify rewrites in Skia, it is hard to do so over Surface$_{\text{Skia}}$. This is mainly because of the hidden surface and clip state of Surface$_{\text{Skia}}$. That is why we introduce a new language called Core$_{\text{Skia}}$ which is a declrative version of Surface$_{\text{Skia}}$. Core$_{\text{Skia}}$ makes surfaces the main value type of the language, and also make explicit the clip state at each draw command. This means neither `Save`/`SaveLayer` are needed to "manage" clip state in Core$_{\text{Skia}}$. This also has the consequence of completely removing `Saves` from Core$_{\text{Skia}}$. `SaveLayers` remain in Core$_{\text{Skia}}$ because they also allow us to combine surfaces in addition to managing the clip state.

Bad paragraph. We still don't know if this paper is verification first or optimization first. Also there are way to many Surface$_{\text{Skia}}$ and Core$_{\text{Skia}}$ in this paragraph.

$\text{Core}_{\text{Skia}}$	s	::=	l
Layer	l	::=	`Empty()`
		|	`Draw(`l`,` g`,` p`,` cl`)`
		|	`SaveLayer(`l`,` l`,` p`)`
Geometry	g	::=	`Rect(...)`
		|	`RRect (...)`
		|	`Path(...)`
		|	`...`
ClipGeometry	cl	::=	`Rect(...)`
		|	`RRect (...)`
		|	`Path(...)`
		|	`Intersect(`cl`,` cl`)`
Paint	p	::=	`Paint(`f`,` st`,` c_f`,` b`)`
Fill	f	::=	`Color(Pixel)`
		|	`LinearGradient(...)`
		|	`...`
Style	st	::=	`Solid`
		|	`Stroke`
Filter	c_f	::=	`IdFilter`
		|	`LumaFilter`
		|	`...`
BlendMode	b	::=	`SrcOver`
		|	`DstIn`
		|	`Src`
		|	`...`

The main syntactical structures of $\text{Core}_{\text{Skia}}$ are Layers. A `Layer` correpsonds to one state of the draw surface. An Empty layer is a draw surface with all the pixels set to the argb color `(0, 0, 0, 0)`. A Draw layer draws a geometry onto the given layer. Just like the Draw* commands in $\text{Surface}_{\text{Skia}}$, how the shape is drawn and blended onto the layer is governed by the Paint. A SaveLayer layer is similiar to a SaveLayer(); ...; Restore(); pair in $\text{Surface}_{\text{Skia}}$.

```
bottom
...
SaveLayer(paint);
top
...
Restore();
```

`SaveLayer(bottom, top, paint)`

Paints behave the same way in $\text{Core}_{\text{Skia}}$ as they do in $\text{Surface}_{\text{Skia}}$. Now that we have syntax for $\text{Core}_{\text{Skia}}$ we can define a denotational semantics over it.

everything after this i sprobably written in the worst way possible

We define a denotation function over `Layers` called $[\![\cdot]\!]_{\text{Layer}}$. Its signature is:

$$[\![\cdot]\!]_{\text{Layer}} : int^2 \rightarrow int^4$$

That is the denotation of Layer is map from 2D points to ARGB pixels. We now define the denotations of the various constructors of `Layer`.

$$[\![\texttt{Empty()}]\!]_{\text{Layer}} = \lambda pt.(0, 0, 0, 0)$$

$[\![\ \texttt{Draw(l, g, (f, sty, cf, bm), cl)}\ ]\!]_{\text{Layer}} = \lambda pt.bm(lpt)(cf(if(styleg)and(clippt)then(fillpt)else(0,0$

I have not added savelayer because it will be to painful to looka t at as is

$$[\![\ \texttt{Empty()}\ ]\!]_{\text{Layer}} = \lambda pt.(0,0,0,0)$$

4 Compiler

5 Evaluation

6 Related Work

7 Conclusion